How LLMs Work: Inference

Jim Fawcett · conversation with Claude · October 18, 2026

PROMPT

> Describe the LLM AI inference process assuming we have the context of the previous prompt and reply.

REPLY

LLM Inference

1. How inference relates to training

Inference reuses the pretraining machinery with three changes:

- **Frozen parameters.** There is no loss, no backpropagation, and no optimizer step. The model function is only evaluated, never updated.
- **One request at a time from the user's view.** Input is a single prompt, not a batch drawn from the corpus. The serving system may still batch many users' requests together for efficiency.
- **Generation instead of scoring.** In training, the next-token distribution at each position is compared against the known next token. In inference there is no known answer, so the model *picks* a token from the distribution and feeds it back in.

The model function, embedding → N layers of attention + MLP → unembedding → softmax, is exactly the same.

2. Preparing the input

- **Assembling the context.** For a chat model, the input is more than the user's message. It is a single token sequence containing:
  - a system prompt
  - the conversation so far
  - any attached documents or tool results
  - a marker saying “assistant turn starts here”
- **Chat template.** Special tokens mark the role boundaries (system / user / assistant). Post-training taught the model what these markers mean, which is why it continues as the assistant instead of, say, writing the user's next line.
- **Tokenizing.** The assembled text is tokenized into integer IDs with the same tokenizer used in training.
- **Context window.** The whole sequence must fit within the model's maximum context length, the longest sequence it was trained or extended to handle. Anything outside the window does not exist for the model.

3. Phase one: prefill

The entire prompt is run through the model in **one forward pass**, all positions in parallel, just as a training sequence is processed. Two things come out of it:

- **The KV cache.** At every layer, the key and value vectors computed for each prompt position are saved. Under the causal mask, a position's keys and values never change once computed, because later tokens can't affect earlier positions. So there is no reason to recompute them.
- **The first output distribution.** Only the distribution at the *last* position is needed; it predicts the first token of the reply. The distributions at earlier positions are computed but discarded.

Prefill is **compute-bound**: lots of parallel matrix work, which GPUs handle efficiently. Its duration largely determines **time to first token**.

4. Phase two: decode loop

Then the model generates one token per step:

1. **Sample** a token from the current next-token distribution (see §5).
2. **Append** it to the sequence. It is also streamed to the user.
3. **Run a forward pass for just that one new token.**
   - In each layer it computes its own query, key, and value.
   - Its query attends to the cached keys and values of *all* previous positions plus itself.
   - Its new key and value are added to the cache.
4. **Repeat with the new distribution** this produces, until one of these happens:
   - an end-of-turn token is generated
   - a stop condition fires
   - the output length limit is reached

Key properties of decode:

- **Inherently sequential.** Token t+1 depends on token t, so steps cannot be parallelized within one response. This is the “autoregressive” in autoregressive generation.
- **Memory-bandwidth-bound.** Each step does relatively little arithmetic but must read *all* the model's weights, plus the whole KV cache, from GPU memory. Throughput in tokens per second is therefore limited mostly by memory bandwidth, not compute. Batching many users' decode steps together amortizes those weight reads, which is a large part of why serving is economical.
- **Cost grows with context.** The KV cache grows linearly with sequence length, and each new token's attention must read all of it. Long contexts consume memory and slow each decode step.

5. Choosing the next token (sampling)

The model outputs a probability distribution. The token chosen from it is decided by a separate **decoding strategy**, not by the model:

- **Greedy.** Always take the most probable token. This is deterministic but tends toward repetitive, flat text.
- **Temperature.** Divide the logits by T before the softmax.
  - T < 1 sharpens the distribution: more predictable output.
  - T > 1 flattens it: more varied output, with more risk of errors.
  - T → 0 approaches greedy.
- **Top-k / top-p (nucleus).** Restrict sampling to the k most likely tokens, or to the smallest set whose probabilities sum to p. This cuts off the long tail of unlikely tokens.

This is why the same prompt can produce different responses. Small early differences compound, because every later token is conditioned on the earlier choices.

6. What the model does and doesn't retain

- **No learning during inference.** Parameters never change. Nothing a user says alters the model.
- **The context is the only working memory.** Each request is independent. In a multi-turn chat, the *entire* conversation is re-sent and re-processed every turn. Products avoid redundant work with **prompt caching**: reusing the KV cache for a prefix that hasn't changed. Features that appear to "remember" across sessions work by inserting stored text into the context.
- **In-context learning.** The model can still pick up new patterns, instructions, or facts from examples in the prompt and apply them within that request. This happens entirely through the forward pass: attention lets later positions use information from earlier ones. Behavior changes; parameters don't.

7. Extensions built on the same loop

- **Reasoning ("thinking") tokens.** Models trained with reinforcement learning to reason generate intermediate tokens before the final answer. These are ordinary generated tokens that become context for what follows. They give the model more forward passes, and so more computation, before it commits to an answer.
- **Tool use.** The model generates a structured tool call (special tokens plus arguments) and stops. The surrounding software, not the model, runs the tool and appends the result to the context. Generation then resumes. Agents are this loop repeated many times.
- **Speculative decoding.** A small, fast draft model proposes several tokens. The large model checks them all in one parallel pass, prefill-style, and keeps the prefix it agrees with. The output is unchanged (it matches what the large model would sample), and decoding is faster.
- **Serving optimizations.**
  - **Quantization:** storing weights in fewer bits, which reduces the memory traffic that dominates decode.
  - **Parallelism:** splitting a model across GPUs when it doesn't fit on one.

- **Continuous batching:** adding and removing users' requests from the batch at every step.

8. Compact statement

> *Inference evaluates the same frozen model function used in training. The prompt, including system instructions, conversation history, and role markers, is tokenized and processed in one parallel prefill pass that builds a key/value cache and yields a distribution for the first output token. The model then generates autoregressively: a decoding strategy picks a token from the distribution, the token is appended, and a single-position forward pass that reads the cache produces the next distribution, repeating until an end token or limit is reached. Nothing is learned. The context window is the model's only working memory, and everything it “knows” in a given request comes from its parameters or from that context.*